

Machine Fault Classifier — Single-Layer Perceptron From Scratch

A foundational Artificial Neural Network (ANN) project built with zero ML libraries (no scikit-learn, no numpy, no pandas — just Python's built-in math). It is designed to demonstrate core Soft Computing techniques from scratch, explaining how a single neuron processes inputs, applies an activation function, and updates its synaptic weights via gradient descent.

1. The Problem

Classify a factory machine's operational status as:

Label	Meaning
0	Healthy Machine (Normal operation)
1	Faulty Machine (Needs immediate maintenance)

The network uses 2 input features, kept simple so the decision boundary can be visualized on a 2D graph:

- x_1 = Temperature Severity (0-10 scale): How much the machine's temperature exceeds normal baselines.
- x_2 = Vibration Intensity (0-10 scale): The intensity of irregular physical rattling or shaking.

2. The Model: Single-Layer Perceptron

This model represents a single artificial neuron. It computes a weighted sum of the sensor inputs and passes that sum through a non-linear activation function.

Step A — Feedforward (Weighted Sum):

$$z = w_1 * x_1 + w_2 * x_2 + b$$

- w_1, w_2 are synaptic weights — representing the strength or importance of each sensor reading to the final decision.
- b is the bias — an intrinsic threshold that shifts the neuron's activation sensitivity.

Step B — Neuron Activation:

$$\text{output} = 1 / (1 + e^{(-z)})$$

We use the Sigmoid activation function to map the raw weighted sum to a continuous value between 0 and 1, representing the network's confidence that a fault is occurring.
Step C — Output Threshold:

- if output ≥ 0.5 -> Neuron fires, predict Faulty (1)
- if output < 0.5 -> Neuron stays dormant, predict Healthy (0)

3. How the Network "Learns" (Backpropagation)

We initialize the synaptic weights (w_1 , w_2) and bias (b) with small random numbers. The network learns the optimal weights over many training cycles (epochs) using gradient descent:

1. Forward Pass: Feed a machine sensor reading through the neuron to get a predicted output.
2. Error Calculation: Compare the prediction to the true target label using a loss function (Binary Cross-Entropy).
3. Backpropagation: Calculate the gradient (derivative of the error with respect to each weight) to determine how much each weight contributed to the error.
4. Weight Update: Adjust the synaptic weights a small step (learning_rate) in the opposite direction of the gradient to minimize future errors.
5. Iterate: Repeat for 2000 epochs.

Over time, the network learns to accurately separate the healthy and faulty data points:

- Epoch 0 | Error: 1.6241
- Epoch 400 | Error: 0.1834
- Epoch 800 | Error: 0.0912
- Epoch 1200 | Error: 0.0543
- Epoch 1600 | Error: 0.0381
- Epoch 1999 | Error: 0.0298

4. Visualization

visualize.py trains the perceptron and plots the results:

-  Green circles = Healthy machines from the training data.
-  Orange triangles = Faulty machines from the training data.
- Shaded background = The neuron's activation space (green = healthy, orange = faulty).
- Black dashed line = The decision boundary — the exact mathematical hyperplane where the network is 50/50 undecided ($w_1 \cdot x_1 + w_2 \cdot x_2 + b = 0$).

5. Files in this project

File	Purpose
perceptron_classifier_scratch.py	The ANN model: dataset, sigmoid, forward pass, backpropagation, and testing.
visualize.py	Trains the model and plots decision_boundary.png (uses matplotlib).
decision_boundary.png	Output visualization map.
README.md	This file.